

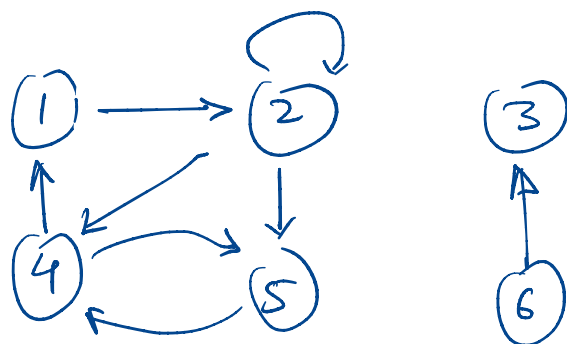
Graph Algorithms

A directed graph or a digraph G is a pair (V, E) where V is a finite set of vertices & E is a binary relationship on V

↳ V is called the vertex set of G
(and its elements are called vertices)

↳ E is called the edge set of G
(and its elements are called edges)

Ex.



$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{(1, 2), (4, 1), (4, 5), (5, 4), (2, 5), (2, 2), (2, 4), (6, 3)\}$$

→ Self loops are possible.

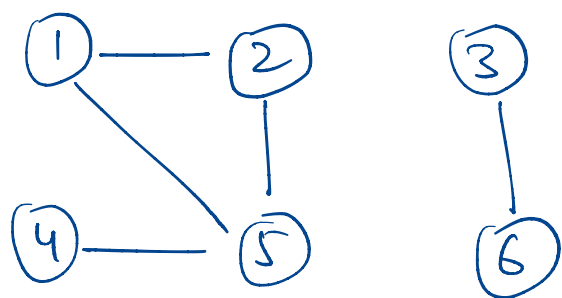
In an undirected graph $G = (V, E)$, the edge set consists of unordered pairs of vertices; rather than ordered.

I.e., an edge is a set $\{u, v\}$ where $u, v \in V$, &

$u \neq v \Rightarrow (u, v) \& (v, u)$ are same.

→ Self loops are forbidden

Ex.



$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{(1, 2), (1, 5), (2, 5), (4, 5), (3, 6)\}$$

- In an undirected graph, if (u, v) is an edge then (u, v) is incident from or leaves vertex u , and is incident to or enters vertex v .
- If (u, v) is an edge of graph $G = (V, E)$ then we say that vertex u is adjacent to vertex v
 - for undirected graph, the adjacency relationship is symmetric.
- The degree of a vertex in an undirected graph is the # of edges incident on it
 - A vertex whose degree is 0 is an isolated vertex

- In a directed graph, we define:

In-degree: # of incoming edges

out-degree: # of outgoing edges

degree = In-degree + out-degree

Representation of graphs

Two standard ways:
 → Adjacency list
 → Adjacency Matrix

both directed & undirected graphs

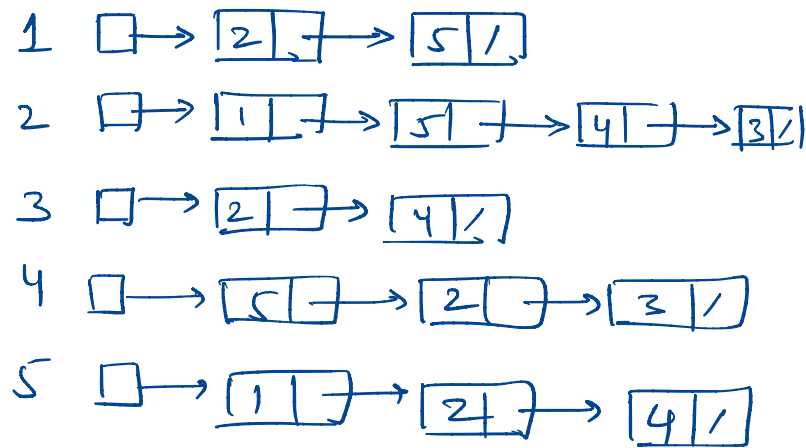
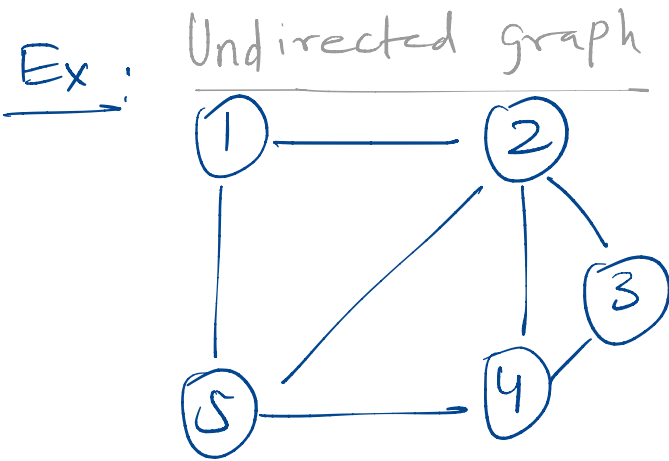
Adjacency Lists: provides a compact representation for sparse graphs — for which $|E|$ is much less than $|V|^2$.

Adjacency Matrix: good for dense graphs — for which $|E|$ is close to $|V|^2$.

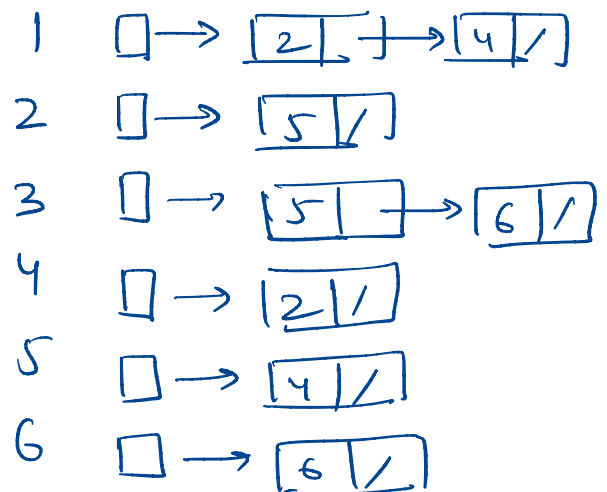
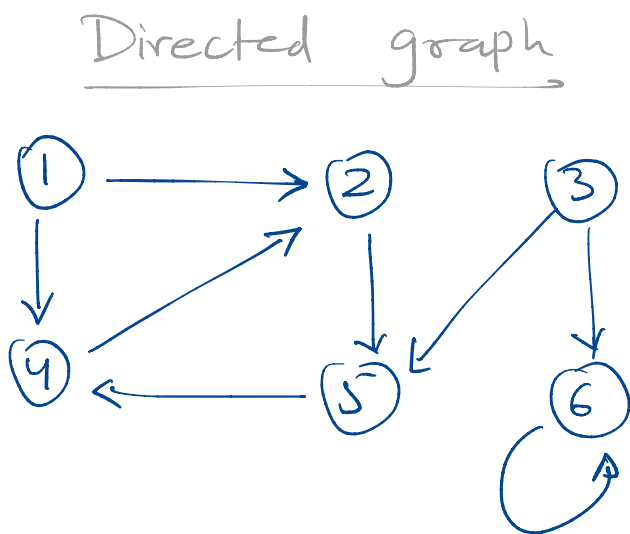
Adjacency List : Representation of a graph

$G = (V, E)$ that contains an array Adj of $|V|$ lists, one for each vertex in V .

→ For each $u \in V$, the adjacency list $Adj[u]$ contains all the vertices v for which there is an edge $(u, v) \in E$.



total # of edges stored = $2|E|$



Total # of edges stored = $|E|$

- Total space requirement $O(V+E)$
- Finding an edge takes $O(V+E)$ time
- Can easily be modified to represent a weighted graph - one with a weight function associated with each edge $w(u,v)$ for edge $(u,v) \in E$.
 - ↳ store $w(u,v)$ with edge (u,v)
- Disadvantage: No quick way to check if edge (u,v) is present!
 - ↳ need to search for it

Adjacency Matrix :

Assume vertices are numbered $1, 2, \dots, |V|$,

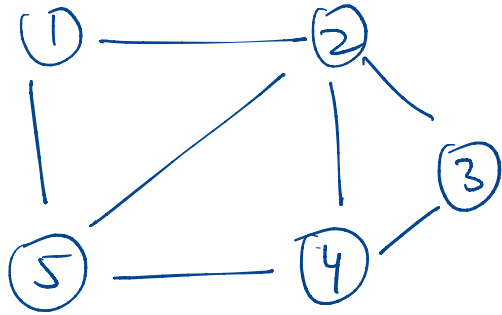
the adjacency matrix is a $|V| \times |V|$ matrix

$A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E \\ 0 & \text{otherwise} \end{cases}$$

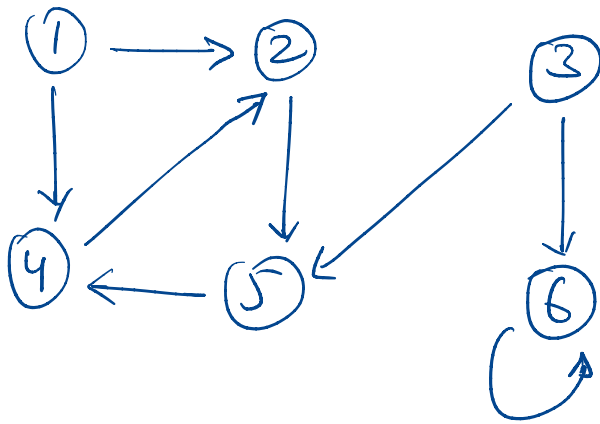
Ex:

Undirected graph



	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Symmetric!



	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

— Required $O(|V|^2)$ memory, independent of the no. of edges.

— For an undirected graph, $A = A^T$
(i.e., A is symmetric)

— Can be modified to store a weighted graph.

$$A_{ij} = \begin{cases} w(i,j) & \text{if } (i,j) \in E \\ \text{NIL} & \text{otherwise} \end{cases}$$

Breadth-first Search (BFS)

Given a graph $G = (V, E)$ and a source vertex s , BFS explores the edges of G systematically to "discover" every vertex that is reachable from s .

→ It computes distance from s to each reachable vertex in G

→ Also produces a Breadth-first Tree with root s that contains all the reachable vertices

— The BFS algorithm uses the Adjacency Lists representation, a queue Q and following vertex attributes to v :

→ $v.color$: WHITE, GRAY, or Black

↙
unprocessed

↓
under
exploration

↘
Processed

→ $v.d$ holds the distance from starting vertex s to v

→ $v.\pi$ is v 's predecessor from vertex s to v . NIL if source or undiscovered

BFS (G, s)

for each vertex $u \in G.V - \{s\}$

$u.color = WHITE$

$u.d = \infty$

$u.\pi = NIL$

$s.color = GRAY$

$s.d = 0$

$s.\pi = NIL$

$S = \emptyset$

Enqueue (Q, s)

while $Q \neq \emptyset$

$u = Dequeue(Q)$

 for each vertex v in $G.Adj[u]$

 if $v.color == WHITE$

$v.color = GRAY$

$v.d = u.d + 1$

$v.\pi = u$

 Enqueue(Q, v)

$u.color = BLACK$

Complexity
 $O(V+E)$

Ex:

